

BLE-Protokoll und Workflow für das Climate-Canary-Counterpart

1 Ziel dieses Dokuments

Dieses Dokument beschreibt den BLE-Workflow für das Counterpart, das mit dem Climate-Canary-Gerät kommuniziert. Es ist absichtlich so geschrieben, dass an jeder Stelle glasklar ist:

- welcher **Service** bzw. welche **Characteristic** verwendet wird,
- welche **UUID** dazugehört,
- in welche **Richtung** Daten fließen,
- welche **Datenstruktur** übertragen wird,
- wie die **Bytes im Little-Endian-Format** aussehen,
- und **wann** welches Paket wohin geschickt werden muss.

2 Grundprinzip

Das Gerät hat zwei Betriebsmodi:

1. **Setup-Modus** für die Erstkonfiguration
2. **Normalbetrieb** für Authentifizierung, Sensorstream, Statusrückmeldungen, Warning-Text und Acknowledgement

3 Wichtiger Hinweis zur Byte-Reihenfolge

Alle Structs werden **packed** und im **Little-Endian-Format** übertragen.

Little Endian

Little Endian bedeutet: **das niederwertigste Byte kommt zuerst.**

Beispiel

```
uint32_t value = 0x12345678;
```

Bytefolge auf BLE:
78 56 34 12

Wichtig

Wenn ein Feld aus mehreren Bytes besteht, darf es nicht byteweise in natürlicher Hex-Schreibweise geschickt werden, sondern muss Little Endian codiert werden.

4 Gerätesuche / Discovery

4.1 Gerätenamen

Das Gerät verwendet je nach Zustand unterschiedliche BLE-Namen:

- **Setup-Modus:** G5T4SETUP
- **Normalbetrieb:** G5T4CC

4.2 Manufacturer Data

Zusätzlich wird Manufacturer Data gesetzt:

- **Setup-Modus:** "00RDY"
- **Normalbetrieb:** "11LCK"

4.3 Advertised Service IDs

Hier ist ein wichtiger Punkt:

- Im **Setup-Modus** wird als advertised service der **Device Setup Service** verwendet.
- Im **Normalbetrieb** wird als advertised service der **Environmental Sensing Service** verwendet.

Setup-Modus: advertised service

94050000-af44-4a64-b339-8b04d5565014

Normalbetrieb: advertised service

181A

Wichtige Klarstellung

Der **Warning Control Service** ist im Normalbetrieb zwar vorhanden und benutzbar, aber **nicht der advertised service**. Das heißt:

- Zum **Scannen** sollte primär auf Name, Manufacturer Data und advertised service geachtet werden.
- Nach dem **Verbinden** sind dann sowohl Environmental Service als auch Warning Control Service vorhanden.

5 Komplette Übersicht aller Services und Characteristics

5.1 Service 1: Device Setup Service

Name: Device Setup Service

Service UUID:

94050000-af44-4a64-b339-8b04d5565014

Characteristic: Setup Config

Name: Setup Config

UUID:

94050001-af44-4a64-b339-8b04d5565014

Zugriff: Write only

Richtung: Counterpart → Gerät

Verwendung: Nur beim allerersten Setup

5.2 Service 2: Environmental Sensing Service

Name: Environmental Sensing Service

Service UUID:

181A

Characteristic: Sensor Packet

Name: Sensor Packet

UUID:

4b8e0001-2581-4c5c-8a61-deb186a46179

Zugriff: Read, Notify

Richtung: Gerät → Counterpart

Verwendung: Live-Sensordaten abonnieren und lesen

Characteristic: Sensor Packet Status

Name: Sensor Packet Status

UUID:

4b8e0002-2581-4c5c-8a61-deb186a46179

Zugriff: Read, Write

Richtung: Counterpart → Gerät

Verwendung: Statusbewertung eines empfangenen Sensorpakets zurückschreiben

Characteristic: Cached Sensor Data

Name: Cached Sensor Data

UUID:

4b8e0003-2581-4c5c-8a61-deb186a46179

Zugriff: Read

Richtung: Gerät → Counterpart

Verwendung: Nach Authentifizierung gespeicherte historische Sensorpakete auslesen

Characteristic: Cached Sensor Data ACK

Name: Cached Sensor Data ACK

UUID:

4b8e0004-2581-4c5c-8a61-deb186a46179

Zugriff: Write, Notify

Richtung: Counterpart → Gerät

Verwendung: Mit `true` das nächste historische Paket anfordern

5.3 Service 3: Warning Control Service

Name: Warning Control Service

Service UUID:

bda70000-24ff-4f28-af24-8293a69561ca

Characteristic: Authentication Packet

Name: Authentication Packet

UUID:

bda70001-24ff-4f28-af24-8293a69561ca

Zugriff: Write

Richtung: Counterpart → Gerät

Verwendung: Pflichtpaket direkt nach dem Connect im Normalbetrieb

Characteristic: Warning Acknowledged

Name: Warning Acknowledged

UUID:

bda70002-24ff-4f28-af24-8293a69561ca

Zugriff: Read, Write, Notify

Richtung: in beide Richtungen

Verwendung: Warning quittieren oder Quittierung vom Gerät mitbekommen

Characteristic: Warning Message Total Length

Name: Warning Message Total Length

UUID:

bda70003-24ff-4f28-af24-8293a69561ca

Zugriff: Read, Write

Richtung: Counterpart → Gerät

Verwendung: Gesamtlänge des kompletten Warning-Text-Streams in Bytes

Characteristic: Warning Message Char Pack

Name: Warning Message Char Pack

UUID:

bda70004-24ff-4f28-af24-8293a69561ca

Zugriff: Read, Write

Richtung: Counterpart → Gerät

Verwendung: Genau ein Zeichen eines Warning-Text-Streams übertragen

Characteristic: Warning Message ACK Request

Name: Warning Message ACK Request

UUID:

bda70005-24ff-4f28-af24-8293a69561ca

Zugriff: Read, Notify

Richtung: Gerät → Counterpart

Verwendung: Gerät fordert die nächste gewünschte Byte-Position des Warning-Textes an

6 Datenstrukturen

6.1 DeviceSetupConfig

Diese Struktur wird auf **Setup Config** geschrieben.

```
struct DeviceSetupConfig {  
    uint8_t measurementInterval;  
    uint32_t deviceId;  
};
```

Feldbeschreibung

- measurementInterval: Messintervall in Sekunden, genau 1 Byte
- deviceId: Geräte-ID, genau 4 Byte, Little Endian

Gesamtgröße

1 + 4 = 5 Byte

6.2 DeviceAuthenticationPacket

Diese Struktur wird auf **Authentication Packet** geschrieben.

```
struct DeviceAuthenticationPacket {  
    uint32_t deviceId;  
    char      roomName[32];  
    uint8_t   roomNameLength;  
};
```

Feldbeschreibung

- deviceId: dieselbe ID wie im Setup
- roomName[32]: Raumname als Bytearray, maximal 32 Byte
- roomNameLength: tatsächliche Länge des verwendeten Namens

Gesamtgröße

4 + 32 + 1 = 37 Byte

6.3 SensorDataPacket

Diese Struktur wird vom Gerät auf **Sensor Packet** und **Cached Sensor Data** bereitgestellt.

```
struct SensorDataPacket {  
    uint32_t timestamp;  
    float    pressure;  
    float    temperature;  
    float    humidity;  
    uint32_t gasResistance;  
};
```

Feldbeschreibung

- **timestamp**: Gerätezeit in Millisekunden seit Boot
- **pressure**: Druck
- **temperature**: Temperatur
- **humidity**: Luftfeuchtigkeit
- **gasResistance**: Gaswiderstand

Gesamtgröße

$4 + 4 + 4 + 4 + 4 = 20$ Byte

6.4 SensorStatusPacket

Diese Struktur wird auf **Sensor Packet Status** geschrieben.

```
struct SensorStatusPacket {  
    uint32_t timestamp;  
    uint16_t statusCode;  
};
```

Feldbeschreibung

- **timestamp**: Zeitstempel des Sensorpakets, auf das sich die Bewertung bezieht
- **statusCode**: 16-Bit-Bitfeld mit 4 Nibbles

Gesamtgröße

$4 + 2 = 6$ Byte

6.5 WarningMessageChunk

Diese Struktur wird auf **Warning Message Char Pack** geschrieben.

```
struct WarningMessageChunk {  
    uint16_t sequenceNumber;  
    char     content;  
};
```

Feldbeschreibung

- **sequenceNumber**: Position des Zeichens im gesamten Warning-Byte-Stream
- **content**: genau das Zeichen an dieser Position

Gesamtgröße

$2 + 1 = 3$ Byte

7 Erster Login / Erstkonfiguration

7.1 Wann dieser Modus gilt

Wenn das Gerät noch nicht eingerichtet wurde, startet es im Setup-Modus.

7.2 Wie das Gerät gefunden wird

Beim Scan sollte auf Folgendes gefiltert werden:

- Name: G5T4SETUP
- Manufacturer Data: "OORDY"
- advertised service: 94050000-af44-4a64-b339-8b04d5565014

7.3 Wohin geschrieben wird

Auf die Characteristic:

```
Setup Config
UUID: 94050001-af44-4a64-b339-8b04d5565014
```

7.4 Was geschrieben wird

Ein DeviceSetupConfig:

```
struct DeviceSetupConfig {
    uint8_t measurementInterval;
    uint32_t deviceId;
};
```

7.5 Beispielwerte

```
measurementInterval = 10
deviceId             = 123456
```

7.6 Hex-Beispiel

```
measurementInterval = 10 decimal = 0A hex
deviceId             = 123456 decimal = 0x0001E240

deviceId Little Endian:
40 E2 01 00

komplettes Setup-Paket:
0A 40 E2 01 00
```

7.7 Erwartetes Verhalten nach dem Write

Nach dem Schreiben dieses Pakets passiert Folgendes:

1. Das Gerät speichert ID und Messintervall persistent.
2. Das Gerät führt einen Neustart aus.
3. Die BLE-Verbindung wird dadurch unterbrochen.

Das ist das **gewollte Verhalten**.

8 Jeder weitere Login / Normalbetrieb

8.1 Wie das Gerät gefunden wird

Beim Scan sollte auf Folgendes gefiltert werden:

- Name: G5T4CC
- Manufacturer Data: "11LCK"
- advertised service: 181A

8.2 Wichtiger Ablauf direkt nach dem Connect

Nach dem Verbinden ist das Gerät **noch nicht authentifiziert**. Vor erfolgreicher Authentifizierung dürfen insbesondere diese Writes nicht erfolgen:

- Sensor Packet Status
- Warning Message Total Length
- Warning Message Char Pack
- Warning Acknowledged
- Cached Sensor Data ACK

Wenn trotzdem geschrieben wird, trennt das Gerät die Verbindung.

9 Authentifizierung

9.1 Wohin geschrieben wird

Auf die Characteristic:

```
Authentication Packet
UUID: bda70001-24ff-4f28-af24-8293a69561ca
```

9.2 Was geschrieben wird

Ein DeviceAuthenticationPacket:

```
struct DeviceAuthenticationPacket {
    uint32_t deviceId;
    char      roomName[32];
    uint8_t   roomNameLength;
};
```

9.3 Beispiel

```
deviceId      = 123456
roomName      = "Lab1"
roomNameLength = 4
```


9.4 Hex-Beispiel

```
deviceId = 123456 = 0x0001E240
Little Endian:
40 E2 01 00

roomName = "Lab1"
ASCII:
4C 61 62 31

auf 32 Byte gepadded:
4C 61 62 31 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00

roomNameLength = 04

komplettes Authentication-Paket:
40 E2 01 00
4C 61 62 31 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
04
```

9.5 Bedeutung

- Die deviceId muss exakt der im Setup gespeicherten ID entsprechen.
- roomNameLength darf nicht größer als 32 sein.
- Nach erfolgreicher Authentifizierung dürfen die weiteren Counterpart-Funktionen benutzt werden.

9.6 Fehlerfälle

- falsche ID → Disconnect
- ungültige Paketlänge → Disconnect
- ungültige Raumlänge → Disconnect

10 Live-Sensordaten

10.1 Woher die Daten kommen

Vom Gerät auf:

```
Sensor Packet
UUID: 4b8e0001-2581-4c5c-8a61-deb186a46179
```

10.2 Wie die Daten gelesen werden

Diese Characteristic sollte **abonniert** werden.

10.3 Datenstruktur

```
struct SensorDataPacket {
    uint32_t timestamp;
    float    pressure;
    float    temperature;
    float    humidity;
    uint32_t gasResistance;
};
```

10.4 Bedeutung

Das Gerät schreibt in diese Characteristic periodisch aktuelle Sensordaten.

10.5 Wichtiger Implementierungshinweis

Der `timestamp` ist **kein Unix-Zeitstempel**, sondern basiert auf `millis()` seit Gerätestart. Deshalb muss das Counterpart den Timestamp **sofort beim Empfang** auf seine eigene Zeitbasis mappen.

10.6 Beispiel

Angenommen:

```
timestamp      = 1000 = 0x000003E8
pressure       = 1013.25 hPa
temperature    = 24.5 C
humidity       = 45.0 %
gasResistance  = 12000
```

Dann ist die Bytefolge abhängig von der IEEE-754-Darstellung der Floats. Die Integer-Felder sind aber sicher:

```
timestamp Little Endian:
E8 03 00 00

gasResistance = 12000 = 0x00002EE0
Little Endian:
E0 2E 00 00
```

11 Statusrückmeldung zu einem Sensorpaket

11.1 Wohin geschrieben wird

Auf:

```
Sensor Packet Status
UUID: 4b8e0002-2581-4c5c-8a61-deb186a46179
```

11.2 Was geschrieben wird

Ein `SensorStatusPacket`:

```
struct SensorStatusPacket {
    uint32_t timestamp;
    uint16_t statusCode;
};
```

11.3 Wofür das dient

Für jedes relevante Sensorpaket kann das Counterpart den Zustand der vier Messgrößen zurück-melden:

- Pressure
- Temperature
- Humidity
- Gas Resistance

11.4 Bitaufbau des statusCode

Bits	0-3	Pressure
Bits	4-7	Temperature
Bits	8-11	Humidity
Bits	12-15	Gas Resistance

11.5 Mögliche Werte pro Nibble

- 0 = valid
- 1 = short invalid low
- 2 = short invalid high
- 3 = long invalid low
- 4 = long invalid high
- 5 = long valid

11.6 Wichtige Semantik

- 1 und 2: kurzfristige Überschreitung
- 3 und 4: länger anhaltende kritische Überschreitung
- 5: langfristig wieder stabil / alles wieder gut

11.7 Beispiel 1: nur Temperatur kritisch hoch

Pressure	=	0
Temperature	=	4
Humidity	=	0
Gas	=	0

Dann ergibt sich:

statusCode	=	0x0040
Little Endian:		
40	00	

Wenn sich dieses Statuspaket auf Timestamp 1000 bezieht:

```
timestamp = 1000 = 0x000003E8
timestamp Little Endian:
E8 03 00 00
```

```
komplettes SensorStatusPacket:
E8 03 00 00 40 00
```

11.8 Beispiel 2: alles wieder gut

```
Pressure    = 5
Temperature = 5
Humidity    = 5
Gas         = 5
```

Dann gilt:

```
statusCode = 0x5555
Little Endian:
55 55
```

11.9 Wichtiger Ablaufhinweis

Der `timestamp` im `SensorStatusPacket` soll sich auf das Sensorpaket beziehen, das bewertet wird. Also nicht irgendein lokaler Zeitpunkt, sondern die Paketzuordnung muss konsistent bleiben.

12 Wann eine Warning ausgelöst werden darf

Eine echte Warning mit Texttransfer soll nur dann gestartet werden, wenn **mindestens eine** Messgröße den Status 3 oder 4 hat.

- Nur 1 oder 2 bedeutet: kurzfristige Auffälligkeit
- 3 oder 4 bedeutet: Warning-Workflow starten

13 Warning-Workflow im Detail

Der Warning-Workflow nutzt vier Characteristics aus dem Warning Control Service:

1. **Warning Message Total Length**
bda70003-24ff-4f28-af24-8293a69561ca
2. **Warning Message Char Pack**
bda70004-24ff-4f28-af24-8293a69561ca
3. **Warning Message ACK Request**
bda70005-24ff-4f28-af24-8293a69561ca
4. danach zusätzlich **Sensor Packet Status**
4b8e0002-2581-4c5c-8a61-deb186a46179

13.1 Mehrere Warntexte

Mehrere Texte werden **nicht getrennt pro Überschreitung** geschickt. Stattdessen müssen **alle Warning-Nachrichten gemeinsam** in einen einzigen Bytestrom geschrieben werden.

13.2 Textformat

Jede einzelne Nachricht wird mit \0 abgeschlossen. Beispiel:

```
"Bitte lueften\0Fenster oeffnen\0"
```

13.3 Schritt 1: Gesamtlänge schreiben

Characteristic

```
Warning Message Total Length
UUID: bda70003-24ff-4f28-af24-8293a69561ca
```

Datentyp

```
uint16_t totalLength
```

Richtung

Counterpart → Gerät

Beispiel

Für den Stream:

```
"Test\0"
```

ergibt sich:

```
ASCII:
54 65 73 74 00

Laenge = 5 = 0x0005
Little Endian:
05 00
```

Also muss auf Warning Message Total Length geschrieben werden:

```
05 00
```

13.4 Schritt 2: erstes Zeichen senden

Characteristic

```
Warning Message Char Pack
UUID: bda70004-24ff-4f28-af24-8293a69561ca
```

Datenstruktur

```
struct WarningMessageChunk {
    uint16_t sequenceNumber;
    char     content;
};
```

Richtung

Counterpart → Gerät

Wichtig

Nach dem Schreiben der Länge wird zuerst **sofort der erste Character mit sequenceNumber = 0** gesendet.

Beispiel

Textstream:

```
"Test\0"
```

erstes Byte ist 'T' = 0x54

```
sequenceNumber = 0 -> 00 00  
content        = 54
```

```
komplettes Paket:  
00 00 54
```

13.5 Schritt 3: ACK Request abonnieren

Characteristic

```
Warning Message ACK Request  
UUID: bda70005-24ff-4f28-af24-8293a69561ca
```

Datentyp

```
uint16_t requestedSequenceNumber
```

Richtung

Gerät → Counterpart

Bedeutung

Das Gerät meldet per Notify, welche **nächste Position** es haben will.

Wichtig

Der Transfer ist **ACK-gesteuert**. Das Counterpart darf den Text **nicht einfach am Stück blind senden**. Stattdessen gilt:

1. Länge schreiben
2. erstes Zeichen mit `sequenceNumber = 0` schreiben
3. Warning Message ACK Request abonnieren
4. bei jeder angeforderten Nummer genau den passenden `WarningMessageChunk` senden

Beispiel

Wenn das Gerät anfordert:

```
01 00
```

dann bedeutet das:

```
requestedSequenceNumber = 1
```

Dann muss das Counterpart das zweite Byte des Textstreams schicken.

Für "Test" ist Byte 1 das 'e' = 0x65.

Dann ist das Paket:

```
sequenceNumber = 1 -> 01 00  
content        = 65
```

Paket:

```
01 00 65
```

13.6 Schritt 4: nach Texttransfer kritischen Status setzen

Wenn der gesamte Warning-Text übertragen wurde, muss über

```
Sensor Packet Status  
UUID: 4b8e0002-2581-4c5c-8a61-deb186a46179
```

ein `SensorStatusPacket` geschrieben werden, in dem **mindestens ein Nibble 3 oder 4** ist. Erst dadurch ist klar, dass jetzt ein kritischer Zustand vorliegt.

13.7 Praktisches Beispiel

Angenommen, Temperatur ist langfristig zu hoch:

```
Pressure    = 0  
Temperature = 4  
Humidity    = 0  
Gas         = 0
```

Dann gilt:

```
statusCode = 0x0040  
Little Endian:  
40 00
```

Mit Timestamp 1000:

```
E8 03 00 00 40 00
```

14 Acknowledgement der Warning

14.1 Characteristic

```
Warning Acknowledged  
UUID: bda70002-24ff-4f28-af24-8293a69561ca
```

14.2 Datentyp

Boolean, 1 Byte

```
false = 00  
true  = 01
```

14.3 Zugriffsrichtung

In beide Richtungen:

- Counterpart kann **true** schreiben, um zu quittieren
- Gerät kann per Notify bzw. Read den Zustand signalisieren

14.4 Wichtiger Ablauf

Eine Quittierung ist fachlich sinnvoll, wenn die Warning bereits vollständig empfangen wurde.

14.5 Beispiel

Wenn das Counterpart quittieren will, schreibt es:

```
01
```

14.6 Wenn der User am Arduino quittiert

Dann kann das Counterpart denselben Zustand erkennen, indem es **Warning Acknowledged** abonniert.

15 Verhalten solange die Warning aktiv ist

Solange eine Warning aktiv ist:

- laufen normale Sensordpakete weiter,
- das Counterpart soll weiter aktuelle Pakete auswerten,
- und weiter passende **SensorStatusPackets** zurückschreiben.

Die Warning unterbricht also nicht den Sensorstream.

16 Warning beenden / Rückkehr zu all-good

Um die Warning fachlich zu beenden, muss ein **SensorStatusPacket** gesendet werden, in dem **alle vier Messgrößen den Wert 5** haben.

16.1 Beispiel

```
Pressure    = 5  
Temperature = 5  
Humidity    = 5  
Gas         = 5
```

Dann ist:


```
statusCode = 0x5555  
Little Endian:  
55 55
```

Mit Timestamp 1000:

```
E8 03 00 00 55 55
```

16.2 Bedeutung

Dieses Paket setzt den Zustand wieder auf *all good* zurück.

16.3 Danach

Im **nächsten** Paket kann sofort wieder entweder

- eine neue Warning gestartet werden,
- oder nur ein kurzfristiger Status 1 / 2 gesetzt werden.

17 Cached Sensor Data / Historische Pakete

Wenn keine Verbindung oder keine Authentifizierung vorliegt, puffert das Gerät Messwerte lokal. Diese Pakete können später abgeholt werden.

17.1 Lesende Characteristic

```
Cached Sensor Data  
UUID: 4b8e0003-2581-4c5c-8a61-deb186a46179
```

17.2 Trigger-Characteristic

```
Cached Sensor Data ACK  
UUID: 4b8e0004-2581-4c5c-8a61-deb186a46179
```

17.3 Datentyp des ACK

Boolean, 1 Byte:

```
true = 01
```

17.4 Ablauf

Nach erfolgreicher Authentifizierung:

1. auf **Cached Sensor Data ACK true** schreiben
2. dann **Cached Sensor Data** lesen
3. für das nächste historische Paket wieder **true** schreiben
4. wieder lesen
5. solange wiederholen, bis keine neuen Pakete mehr kommen

17.5 Welche Datenstruktur gelesen wird

```
struct SensorDataPacket {
    uint32_t timestamp;
    float    pressure;
    float    temperature;
    float    humidity;
    uint32_t gasResistance;
};
```

Also exakt dieselbe Struktur wie bei den Live-Sensorpaketen.

17.6 Beispiel-ACK

```
01
```

17.7 Wichtig

Auch hier sollte der Timestamp sofort auf die Zeitbasis des Counterparts gemappt werden.

18 Empfohlener Gesamtworkflow

18.1 A. Setup-Fall

1. Scannen nach
 - Name: G5T4SETUP
 - Manufacturer Data: "00RDY"
 - advertised service: 94050000-af44-4a64-b339-8b04d5565014
2. Verbinden
3. auf Setup Config (94050001-af44-4a64-b339-8b04d5565014) ein DeviceSetupConfig schreiben
4. Disconnect / Neustart abwarten

18.2 B. Normalbetrieb

1. Scannen nach
 - Name: G5T4CC
 - Manufacturer Data: "11LCK"
 - advertised service: 181A
2. Verbinden
3. **sofort authentifizieren** über

```
Authentication Packet
UUID: bda70001-24ff-4f28-af24-8293a69561ca
```

4. optional historische Pakete lesen über

- `Cached Sensor Data ACK`
 - `Cached Sensor Data`
5. `Sensor Packet` abonnieren
 6. empfangene Sensorpakete auswerten
 7. zugehörige `SensorStatusPackets` auf `Sensor Packet Status` zurückschreiben
 8. falls mindestens ein Status 3 oder 4 ist:
 - (a) alle Warning-Texte zu einem gemeinsamen Null-terminierten Byte-Stream zusammenbauen
 - (b) Länge auf `Warning Message Total Length` schreiben
 - (c) ersten Chunk mit `sequenceNumber = 0` auf `Warning Message Char Pack` schreiben
 - (d) `Warning Message ACK Request` abonnieren
 - (e) alle angeforderten Chunks nacheinander schicken
 - (f) anschließend kritisches `SensorStatusPacket` schreiben
 9. `Warning Acknowledged` abonnieren, um Quittierungen in beide Richtungen zu sehen
 10. wenn alles wieder gut ist: `statusCode = 0x5555` senden

19 Typische Fehlerquellen

- falsches Endian bei `uint16_t` oder `uint32_t`
- `roomNameLength` passt nicht zu den tatsächlichen Bytes
- `roomName` nicht sauber auf 32 Byte gepadded
- Warning-Bytes werden ohne ACK-Steuerung blind am Stück gesendet
- `SensorStatusPacket` verwendet falschen oder unpassenden Timestamp
- Warning wird fachlich gestartet, obwohl kein Status 3 oder 4 gesetzt ist
- beim Scan fälschlich nach dem Warning-Control-Service suchen, obwohl im Normalbetrieb nur 181A advertised wird

20 Kurzreferenz: Was wohin geschickt wird

Setup

- **Characteristic:** Setup Config
- **UUID:** 94050001-af44-4a64-b339-8b04d5565014
- **Payload:** DeviceSetupConfig

Login

- **Characteristic:** Authentication Packet
- **UUID:** bda70001-24ff-4f28-af24-8293a69561ca
- **Payload:** DeviceAuthenticationPacket

Live-Daten

- **Characteristic:** Sensor Packet
- **UUID:** 4b8e0001-2581-4c5c-8a61-deb186a46179
- **Payload vom Gerät:** SensorDataPacket

Statusantwort

- **Characteristic:** Sensor Packet Status
- **UUID:** 4b8e0002-2581-4c5c-8a61-deb186a46179
- **Payload:** SensorStatusPacket

Warning-Länge

- **Characteristic:** Warning Message Total Length
- **UUID:** bda70003-24ff-4f28-af24-8293a69561ca
- **Payload:** uint16_t

Warning-Chunk

- **Characteristic:** Warning Message Char Pack
- **UUID:** bda70004-24ff-4f28-af24-8293a69561ca
- **Payload:** WarningMessageChunk

Warning-ACK-Request vom Gerät

- **Characteristic:** Warning Message ACK Request
- **UUID:** bda70005-24ff-4f28-af24-8293a69561ca
- **Payload:** uint16_t

Warning quittieren

- **Characteristic:** Warning Acknowledged
- **UUID:** bda70002-24ff-4f28-af24-8293a69561ca
- **Payload:** Boolean (00 oder 01)

Historische Daten holen

- **Trigger:** Cached Sensor Data ACK
UUID: 4b8e0004-2581-4c5c-8a61-deb186a46179
Payload: Boolean 01
- **Lesen von:** Cached Sensor Data
UUID: 4b8e0003-2581-4c5c-8a61-deb186a46179
Payload: SensorDataPacket